S25-47-D-InterviewDen

Project Team

Hamza Saeed	21I-0671
Waleed Saeed	21I-1672
Asad Ullah Shah	21I-0545

Session 2021 - 2025

Supervised by

Dr. Mehreen Alam

Department of Computer Science

National University of Computer and Emerging Sciences
Islamabad, Pakistan

June, 2025

Contents

1 Introduction **1**

1.1 Existing Solutions . . . 1

1.1.1 Resume Screening Solutions . . . 1

1.1.2 Technical Assessment Platforms . . . 2

1.1.3 AI Interview Systems . . . 2

1.1.4 End-to-End Platforms . . . 2

1.1.5 Key Market Gaps . . . 2

1.1.6 InterviewDen's Differentiation . . . 3

1.2 Problem Statement . . . 6

1.3 Scope . . . 6

1.4 Modules . . . 6

1.5 Modules . . . 7

1.5.1 Module 1: AI Resume Screening System . . . 7

1.5.2 Module 2: AI Assessment Generation System . . . 7

1.5.3 Module 3: AI Interview System . . . 8

1.5.4 Module 4: Comprehensive Scoring and Analytics System . . . 9

1.5.5 Module 5: Integration and Platform Management . . . 9

1.6 Work Division . . . 11

2 Project Requirements **13**

2.1 Use-case/Event Response Table/Storyboarding . . . 13

2.1.1 Detailed Use Case Diagrams . . . 16

2.2 Functional Requirements . . . 28

2.2.1 Module 1 . . . 28

2.2.2 Module 2 . . . 28

2.2.3 Module 3 . . . 29

2.2.4 Module 4 . . . 29

2.2.5 Module 5 . . . 30

2.3 Non-Functional Requirements . . . 31

2.3.1 Reliability . . . 31

2.3.2 Usability . . . 31

2.3.3 Performance . . . 32

2.3.4 Security 32
2.3.5 Scalability 33
2.3.6 Compatibility 33
2.3.7 Maintainability 33

3 System Overview **35**
3.1 Architectural Design 36
3.2 Data Design 37
3.3 Domain Model 38
3.4 Design Models [UPTO THE CURRENT ITERATION ONLY] 39

4 Implementation and Testing [UPTO THE CURRENT ITERATION ONLY] **41**
4.1 Algorithm Design 41
4.1.1 Data Processing Pipeline 41
4.1.2 Core Matching Algorithm 42
4.1.2.1 Hard Facts Matching 42
4.1.2.2 LLM-Enhanced Contextual Analysis 42
4.1.2.3 Composite Scoring 42
4.1.3 Algorithm Advantages 42
4.1.4 Implementation Considerations 43
4.2 External APIs/SDKs 46
4.3 Testing Details 46
4.3.1 Unit Testing 46
4.3.2 Integration Testing 46
4.3.3 Prototype Validation 47
4.3.3.1 Diverse Resume-Job Testing 47
4.3.3.2 Algorithm Effectiveness Assessment 47

References **49**

A Appendices **51**
A.1 Appendix A 51
A.1.1 Use Case Diagram 51
A.2 Appendix B 53
A.3 Appendix C 55
A.3.1 Architecture Design 55
A.4 Appendix D 56
A.4.1 System Sequence Diagram 56

List of Figures

2.1 Use Case Diagram 15

3.1 System Architecture 36
3.2 Data Design Diagram 37
3.3 Domain Model Diagram 38
3.4 SSD 1 39

4.1 Converting Job Description to JSON 43
4.2 Resume Matching with Job Description 44
4.3 Resume Score Criteria 44
4.4 Job Description JSON 44
4.5 Resume JSON 44
4.6 Resume Score JSON 45

A.1 Use Case Diagram 52
A.2 Domain Model Diagram 54
A.3 Architecture Diagram 55
A.4 SSD 1 56
A.5 SSD 2 56
A.6 SSD 3 57
A.7 SSD 4 57
A.8 SSD 5 57
A.9 SSD 6 57
A.10 SSD 7 58
A.11 SSD 8 58
A.12 SSD 9 58
A.13 SSD 10 58
A.14 SSD 11 58

List of Tables

1.1 Competitor Analysis: AI Hiring for Tech-Based Industries 5
1.2 Work Divivsion 11

2.1 Use Case 1: Upload and Optimize Resume 16
2.2 Use Case 2: Search and Apply for Jobs 17
2.3 Use Case 3: Complete Skills Assessment 18
2.4 Use Case 4: Participate in AI Interview 19
2.5 Use Case 5: Track Application Status 20
2.6 Use Case 6: Post Job Listing 21
2.7 Use Case 7: Configure AI Assessment 22
2.8 Use Case 8: Review Screened Candidates 23
2.9 Use Case 9: Analyze Test Results 24
2.10 Use Case 10: Review AI Interview Results 25
2.11 Use Case 11: Generate Recruitment Analytics 27

Chapter 1

Introduction

Hiring in the IT industry has long been a challenging and time-consuming process. From manually reviewing countless resumes to conducting lengthy interviews, companies often struggle to find the right candidates efficiently. Human biases can sometimes influence decisions, making it harder to ensure fairness. On the other hand, job seekers frequently face uncertainty—waiting for feedback and wondering how they performed during interviews. To address these issues, we propose the AI-Driven Job Interview Analyzer—a smart system designed to streamline hiring, provide fair and objective evaluations, and deliver timely feedback to both employers and candidates. By reducing manual effort and improving decision-making, this system aims to make the hiring process faster, more accurate, and more transparent for everyone involved.

1.1 Existing Solutions

The recruitment industry has undergone substantial transformation with the integration of artificial intelligence technologies over the past decade. As organizations face increasing pressure to efficiently identify qualified candidates from large applicant pools while improving diversity and candidate experience, numerous AI-driven solutions have emerged to address different aspects of the hiring process. By understanding the state of existing solutions, we establish the foundation for how InterviewDen addresses persistent gaps in the market and delivers meaningful improvements to the recruitment ecosystem.

1.1.1 Resume Screening Solutions

- **Traditional ATS** (Workday, Taleo) offer efficient volume processing but rely on simplistic keyword matching with high false negative rates

- **AI-Enhanced Screeners** (Ideal, HireVue) provide more sophisticated matching but may perpetuate biases and lack transparency
- **Resume Optimizers** (Jobscan, VMock) offer improvement suggestions but focus on gaming systems rather than substantive enhancements

1.1.2 Technical Assessment Platforms

- **Coding Platforms** (HackerRank, Codility) provide standardized evaluation but often emphasize algorithmic puzzles over practical skills
- **Skills Assessments** (TestGorilla, Vervoe) cover diverse domains but typically offer generic tests not tailored to specific positions
- **Simulation Tools** (DevSkiller, Qualified) create job-relevant tasks but face standardization challenges

1.1.3 AI Interview Systems

- **Asynchronous Video** (HireVue, Spark Hire) enable convenient screening but lack natural conversation flow
- **Conversational AI** (Mya, Paradox) offer immediate engagement but have limited depth
- **Comprehensive Systems** (Interviewer.AI, Talview) provide multi-faceted evaluation but struggle with language diversity and transparency

1.1.4 End-to-End Platforms

- **Full-Service Platforms** (LinkedIn Talent, iCIMS) unify workflows but offer less specialized capabilities
- **AI-First Solutions** (Eightfold.ai, Phenom) feature advanced matching but require complex implementation

1.1.5 Key Market Gaps

- Fragmented stages rather than integrated experiences
- Limited actionable feedback for candidates

- Black-box evaluation lacking transparency
- Generic rather than customized assessments
- Imbalance between efficiency and candidate experience
- Inadequate bias detection and mitigation

1.1.6 InterviewDen's Differentiation

InterviewDen addresses these gaps through a unified AI framework, dual-portal approach, continuous feedback, transparent assessment criteria, dynamic customization, comprehensive bias detection, and integrated analytics that connect all recruitment stages for both efficiency and improved candidate experience.

Table 1.1: Competitor Analysis: AI Hiring for Tech-Based Industries

Feature/Platform	**InterviewDen (Tech-Focused)**	**HireVue**	**Micro1**	**Interviewer.AI**	**Talently.ai**	**Jobylon**
Tech Industry Focus	100% tech-focused hiring	General hiring across industries	Specializes in tech hiring (engineers, AI trainers)	General hiring across industries	Tech hiring with AI interviews	General hiring across industries
AI Resume Screening & Feedback	AI-powered, tech-specific screening	General AI-driven assessments	No resume screening (focuses on expert vetting)	Maps resumes to job descriptions	No structured resume screening	No AI resume screening
Technical MCQ & Coding Tests	MCQs, text-based questions, auto-scored	Includes technical assessments but not industry-specific	No structured coding tests (focus on pre-vetted engineers)	Assesses technical and soft skills	No structured coding tests	No coding tests
AI-Powered Video Interviews	AI-simulated interviews for tech roles	Structured AI video interviews	No AI video interviews	AI-powered smart video interviews	AI conducts live, conversational interviews	No AI video interviews
Final Candidate Ranking (Tech-Specific Weightage)	Custom ranking (e.g., 60% tech, 10% resume, 30% interview)	No customizable ranking	No ranking system (focus on sourcing engineers)	Stack ranks candidates	No ranking system	No AI-driven ranking
AI-Powered Candidate Feedback	Yes, at all stages	No feedback system	No feedback system	No feedback system	No feedback system	No feedback system
High-Volume Tech Hiring	Automates screening & ranking for scale	Supports high-volume hiring but not tech-specific	No (focuses on hiring top-tier engineers only)	Automates pre-screening for large pools	No (focuses on live AI interviews)	Supports high-volume hiring, but not AI-driven
Freelance & Global Tech Hiring	Yes	No	Yes, hires and manages international engineers	No	No	Scalable for global hiring, but not AI-driven
Customization & Workflow Automation	Custom hiring stages for tech roles	Limited customization	No workflow automation (focus on pre-vetting)	Limited customization	No workflow automation	Strong customization & workflow automation

1.2 Problem Statement

The hiring process in the IT industry is plagued by inefficiencies:

• Time-Consuming Process: Companies manually review hundreds of resumes, leading to significant delays in identifying suitable candidates.

• Human Bias and Errors: The manual process introduces subjective biases, and candidates may misrepresent themselves through fake or misleading information on their resumes.

• Lack of Timely Feedback: Candidates often do not receive immediate feedback, leading to frustration and uncertainty in the recruitment pipeline.

• Resource Wastage: Companies waste valuable time and resources during interviews, which could be spent more productively.

This project aims to streamline the hiring process by automating key components of candidate evaluation through AI, offering both faster decisions and a better experience for candidates.

1.3 Scope

Our AI-driven interviewer streamlines recruitment by automating resume screening, test creation, and interview monitoring, ensuring a faster and fairer hiring process.

Designed for students, job seekers, and companies, it leverages AI for objective evaluation, real-time assessments, and customizable scoring.

With dedicated company and candidate portals, it simplifies job applications, talent acquisition, and decision-making, making hiring more efficient, transparent, and unbiased.

1.4 Modules

This section describes the key functional units or modules of the project, each designed to perform a specific task. The breakdown of the project into modules ensures that each part of the system is manageable and addresses specific objectives. Each module should be described in terms of its functionality and how it interacts with other modules in the system.

The modules should be logically organized to reflect the overall system workflow, ensuring that each module description is comprehensive but concise.

1.5 Modules

1.5.1 Module 1: AI Resume Screening System

Objective: Automate the evaluation of candidate resumes based on job requirements

Features:

1. **Resume Parsing Engine**
   - Convert diverse CV formats (PDF, DOCX, JPG) into standardized data structure
   - Extract key information (skills, experience, education, projects)
2. **Job-Resume Matching System**
   - Match extracted resume data against job description requirements
   - Calculate comprehensive match percentage and fit score
   - Identify key skills present and missing for the position
3. **Fraud Detection System**
   - Identify inconsistencies and potential misrepresentations
   - Flag suspicious patterns or unlikely qualifications
   - Validate claimed certifications and education when possible
4. **Resume Recommendation Engine**
   - Provide personalized improvement suggestions for candidates
   - Highlight critical missing skills for specific positions
   - Suggest format and content optimizations

1.5.2 Module 2: AI Assessment Generation System

Objective: Create tailored tests that accurately evaluate candidates' skills for specific positions

Features:

1. **Dynamic Question Generation**
   - AI-powered creation of job-specific questions

- Support for multiple formats

2. **Test Configuration Interface**
    - Customizable test parameters (duration, difficulty, question types)
    - Template management for recurring positions
    - Optional manual question addition and editing
3. **Progressive Assessment Engine**
    - Adaptive difficulty based on candidate performance
    - Skill-specific assessment sections
    - Time management and efficiency metrics
4. **Automated Scoring System**
    - Objective evaluation of technical responses
    - Natural language processing for text-based answers
    - Comparative scoring against benchmark responses

1.5.3 Module 3: AI Interview System

Objective: Conduct and analyze automated interviews to evaluate technical knowledge and soft skills

Features:

1. **Conversational Interview Interface**
    - Natural language dialogue management
    - Context-aware follow-up questions
    - Support for both text and voice-based interactions
2. **Technical Skill Evaluation**
    - Domain-specific knowledge assessment
    - Problem-solving capability measurement
    - Logical reasoning and analytical thinking evaluation
3. **Communication Analysis Engine**
    - Speech pattern and clarity assessment

- Body language and eye contact analysis (video mode)
- Confidence and presentation skills evaluation

4. **Real-time Monitoring Dashboard**
   - Interview progress tracking for administrators
   - Performance indicators during live sessions
   - Technical issue detection and resolution

1.5.4 Module 4: Comprehensive Scoring and Analytics System

Objective: Provide data-driven evaluation and insights across the entire recruitment process

Features:

1. **Multi-dimensional Scoring Framework**
   - Weighted evaluation across all assessment stages
   - Customizable scoring priorities for different roles
   - Comparative ranking among candidate pool
2. **Performance Analytics Dashboard**
   - Visual representation of candidate strengths/weaknesses
   - Skill gap analysis with industry benchmarks
   - Historical performance tracking
3. **Hiring Process Optimization**
   - Recruitment funnel analytics
   - Predictive models for candidate success

1.5.5 Module 5: Integration and Platform Management

Objective: Ensure seamless operation and management of the entire system

Features:

1. **User Management System**
   - Role-based access control for different stakeholders

- Customizable permissions and authentication
- Profile management for candidates and company users

2. **Dual Portal Interface**
    - Company portal for job posting and candidate management
    - Candidate portal for applications and feedback
3. **Notification and Communication System**
    - Automated status updates and reminders
    - Feedback delivery to candidates
4. **API and Third-party Integration**
    - Integration with applicant tracking systems
    - Calendar synchronization for scheduling
    - Data export capabilities for external analysis

1.6 Work Division

Table 1.2: Work Divivsion

Name	**Registration**	**Responsibility / Module / Feature**
Hamza Saeed	21i-0671	Candidate Portal Frontend Resume Screening AI Scoring System Analytics Dashboard System Integration User Testing
Waleed Saeed	21i-1672	Data Preprocessing Resume Screening AI Test Creation AI Interview AI AI Integration Testing
Asad Ullah Shah	21i-0545	UI/UX Design Company Portal Frontend Test Creation AI Backend Development Cloud Deployment Testing

Chapter 2

Project Requirements

2.1 Use-case/Event Response Table/Storyboarding

1. **Upload and Optimize Resume**
   Allows candidates to upload their resume for AI analysis and improvement suggestions to increase match rates with relevant job positions.

2. **Search and Apply for Jobs**
   Enables candidates to browse available positions with filtering options and submit applications with personalized match scoring.

3. **Complete Skills Assessment**
   Facilitates customized technical and soft skills evaluation through various question formats tailored to specific job requirements.

4. **Participate in AI Interview**
   Provides automated preliminary interviews that evaluate technical knowledge, communication skills, and problem-solving abilities.

5. **Track Application Status**
   Offers candidates real-time visibility into their application progress across all stages with personalized feedback.

6. **Post Job Listing**
   Allows companies to create detailed job postings with specific skill requirements, assessment parameters, and application workflows.

7. **Configure AI Assessment**
   Enables employers to customize skill evaluation tests with specific difficulty levels, question types, and scoring criteria.

8. **Review Screened Candidates**
   Presents companies with AI-ranked applicants featuring match scores, skill alignment, and resume analysis findings.

9. **Analyze Test Results**
   Provides detailed analysis of candidate test performance with comparative metrics across key competencies.

10. **Review AI Interview Results**
    Delivers automated scoring of candidate interviews with analysis of technical accuracy, communication skills, and problem-solving.

11. **Generate Recruitment Analytics**
    Creates comprehensive metrics on hiring efficiency, time-to-hire, candidate quality, and process effectiveness.

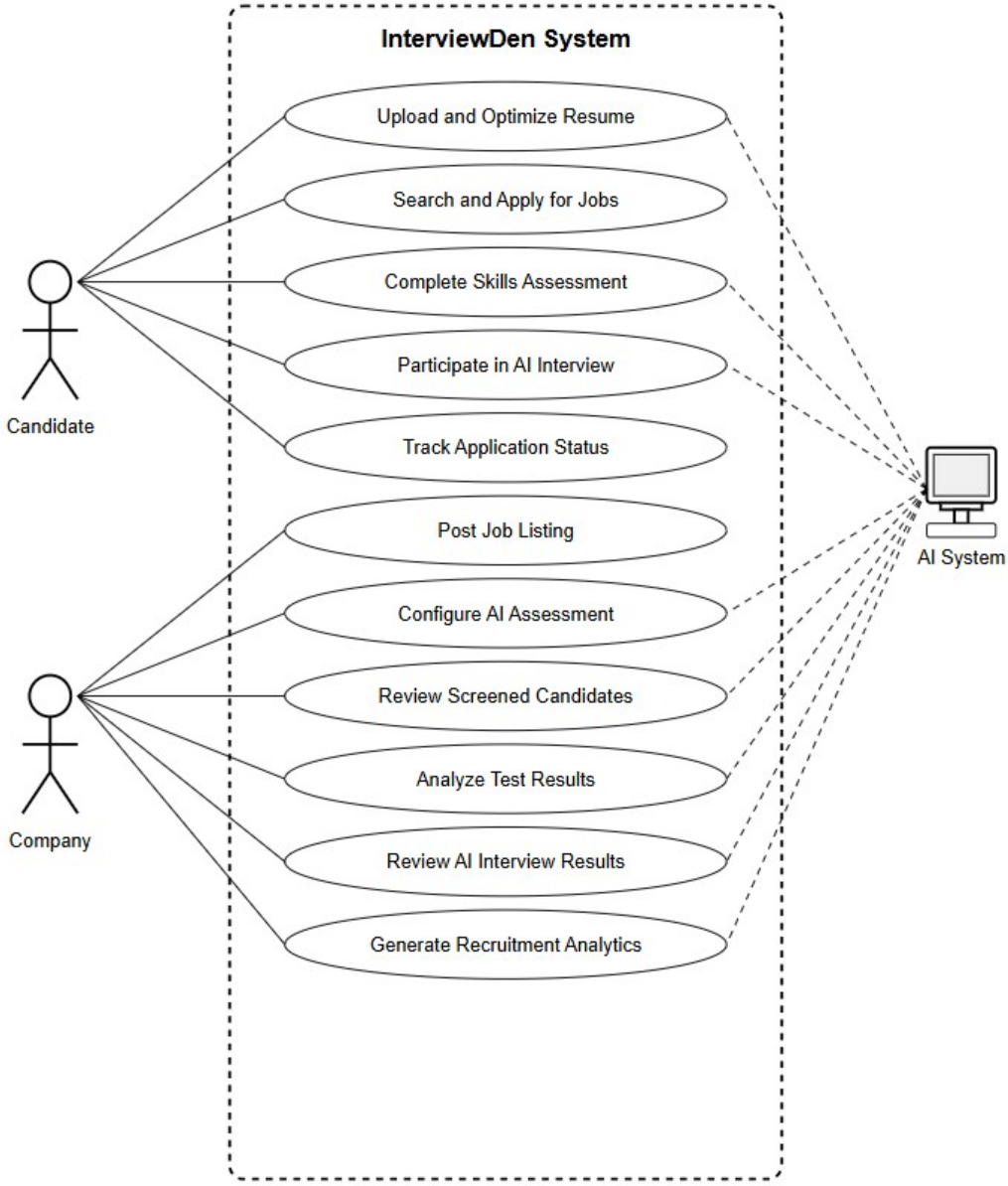

Figure 2.1: Use Case Diagram

2.1.1 Detailed Use Case Diagrams

Building upon the use cases listed in the previous figure, these are the detailed descriptions for each use case.

Use Case 1

Use Case	**Upload and Optimize Resume**
Primary Actor	Candidate
Stakeholders	Candidate, AI System
Preconditions	Candidate registered and logged in
Main Success Scenario	1. Upload resume 2. Parse resume information 3. Analyze resume content 4. Present parsed information 5. Confirm or edit information 6. Provide optimization suggestions 7. Apply suggested changes 8. Save optimized resume
Extensions	• Unsupported file format • Parsing fails • Resume well-optimized
Special Requirements	• Resume parsing must maintain 95% accuracy for key fields • Optimization suggestions must be tailored to industry standards • Process must complete within 30 seconds

Table 2.1: Use Case 1: Upload and Optimize Resume

Use Case 2

Use Case	**Search and Apply for Jobs**
Primary Actor	Candidate
Stakeholders	Candidate, Company
Preconditions	Profile created and resume uploaded
Main Success Scenario	1. Access job search 2. Enter search criteria 3. Display matching jobs 4. Filter results 5. Select job listing 6. View job details 7. Apply to job 8. Pre-fill application 9. Confirm application 10. Submit application
Extensions	• No matching jobs • Low match percentage • Missing profile information
Special Requirements	• Search results must load within 3 seconds • Match percentage calculation must consider at least 10 different factors

Table 2.2: Use Case 2: Search and Apply for Jobs

Use Case 3

Use Case	**Complete Skills Assessment**
Primary Actor	Candidate
Stakeholders	Candidate, Company, AI System
Preconditions	Invited to assessment
Main Success Scenario	1. Receive assessment invitation 2. Schedule assessment 3. Start assessment 4. Complete various question types 5. System monitors for cheating 6. Evaluate code execution 7. Analyze response quality 8. Generate skills profile 9. Receive performance feedback 10. Company notified
Extensions	• Rescheduling request • Cheating detected • Code execution timeout
Special Requirements	• Assessment environment must support multiple programming languages • System must handle network interruptions with auto-save functionality • Assessment scoring must complete within 2 minutes of submission

Table 2.3: Use Case 3: Complete Skills Assessment

Use Case 4

Use Case	**Participate in AI Interview**
Primary Actor	Candidate
Stakeholders	Candidate, Company, AI System
Preconditions	Passed initial screening
Main Success Scenario	1. Receive AI interview invitation 2. Schedule interview 3. Join interview environment 4. Receive interview process overview 5. Respond to technical and behavioral questions 6. AI analyzes responses 7. AI adapts questioning 8. Maintain conversation flow 9. Complete interview 10. Receive performance feedback
Extensions	• Technical communication issues • Unclear responses • Need for question clarification
Special Requirements	• Speech recognition accuracy must exceed 95% • System must handle accents and non-native speakers • Interview session must record and store for company review

Table 2.4: Use Case 4: Participate in AI Interview

Use Case 5

Use Case	**Track Application Status**
Primary Actor	Candidate
Stakeholders	Candidate, Company
Preconditions	Submitted at least one application
Main Success Scenario	1. Access application dashboard 2. View application statuses 3. Select specific application 4. View application progress 5. Review stage feedback 6. Check upcoming steps 7. Receive improvement recommendations 8. Get status change notifications
Extensions	• Application rejected • Low assessment scores • No recent updates
Special Requirements	• Status updates must be reflected within 1 hour of company changes • Notification system must support email, SMS, and in-app alerts

Table 2.5: Use Case 5: Track Application Status

Use Case 6

Use Case	**Post Job Listing**
Primary Actor	Company
Stakeholders	Company, Candidates
Preconditions	Company profile completed
Main Success Scenario	1. Access job creation interface 2. Enter job details 3. Define required skills 4. Set compensation information 5. Receive job description suggestions 6. Configure screening criteria 7. Set up assessment parameters 8. Define application workflow 9. Preview job listing 10. Publish listing 11. System indexes job
Extensions	• Potential bias detection • Incomplete assessment configuration • Save as draft
Special Requirements	• Job listing creation form must auto-save every 60 seconds • Publishing process must complete within 5 seconds

Table 2.6: Use Case 6: Post Job Listing

Use Case 7

Use Case	**Configure AI Assessment**
Primary Actor	Company
Stakeholders	Company, Candidates, AI System
Preconditions	Job listing created
Main Success Scenario	1. Access assessment configuration 2. Select assessment types 3. Define technical skills to test 4. Generate suggested questions 5. Review and customize questions 6. Set difficulty levels 7. Configure time limits and scoring 8. Add custom questions 9. Validate assessment 10. Activate assessment 11. Link to job workflow
Extensions	• Insufficient job details • Assessment imbalance detected • Save as template
Special Requirements	• Question bank must be regularly updated with industry-relevant content • Assessment configuration must support multiple languages • System must ensure assessment difficulty matches job seniority level

Table 2.7: Use Case 7: Configure AI Assessment

Use Case 8

Use Case	**Review Screened Candidates**
Primary Actor	Company
Stakeholders	Company, Candidates
Preconditions	Candidates applied for position
Main Success Scenario	1. Access applicant dashboard 2. View candidates by match percentage 3. Filter candidates 4. Select candidate for detailed review 5. Display comprehensive profile 6. Highlight matching skills 7. Identify potential inconsistencies 8. Review AI candidate evaluation 9. Make hiring decision 10. Update candidate status
Extensions	• No qualified candidates • Resume verification needed • Bulk candidate actions
Special Requirements	• Dashboard must load initial candidate list within 2 seconds • AI screening must achieve 90% agreement with human recruiters • System must comply with anti-discrimination regulations

Table 2.8: Use Case 8: Review Screened Candidates

Use Case 9

Use Case	**Analyze Test Results**
Primary Actor	Company
Stakeholders	Company, Candidates, AI System
Preconditions	Candidates completed assessments
Main Success Scenario	1. Access test results dashboard 2. View overall performance metrics 3. Filter candidates 4. View comparative analysis 5. Select candidate for detailed review 6. Display test performance breakdown 7. Highlight strengths and weaknesses 8. Review code quality metrics 9. Analyze specific answers 10. Make hiring decision 11. Update candidate status
Extensions	• Unusual test patterns • Code plagiarism detected • Disagree with AI scoring
Special Requirements	• Performance analysis must include benchmarking against industry averages • System must analyze response patterns for insight into problem-solving approaches • Results must be visualized with clear graphics for quick understanding

Table 2.9: Use Case 9: Analyze Test Results

Use Case 10

Use Case	**Review AI Interview Results**
Primary Actor	Company
Stakeholders	Company, Candidates, AI System
Preconditions	Candidates completed AI interviews
Main Success Scenario	1. Access interview results dashboard 2. View candidates by interview performance 3. Select specific candidate 4. Review interview transcript 5. Analyze technical accuracy 6. Assess communication effectiveness 7. Identify behavioral indicators 8. Replay interview segments 9. Review AI interview scoring 10. Make hiring decision 11. Update candidate status
Extensions	• Technical interview issues • Conflicting behavioral indicators • Disagree with AI assessment
Special Requirements	• Video playback must support variable speed and closed captioning • Interview analysis must be available within 10 minutes of completion • System must detect and flag potential biases in AI evaluation

Table 2.10: Use Case 10: Review AI Interview Results

Use Case 11

Use Case	**Generate Recruitment Analytics**
Primary Actor	Company
Stakeholders	Company, AI System
Preconditions	Company has active or completed recruitment processes
Main Success Scenario	1. Company administrator accesses analytics dashboard 2. System displays high-level recruitment metrics 3. Administrator selects specific time period for analysis 4. System generates comprehensive recruitment funnel visualization 5. System provides metrics on time-to-hire and cost-per-hire 6. System analyzes effectiveness of different screening stages 7. System identifies bottlenecks in recruitment process 8. System compares current metrics against historical benchmarks 9. Administrator can drill down into specific positions or departments 10. System generates recommendations for process improvement 11. Administrator can export reports in various formats
Extensions	• Insufficient data: System notes limitations of analysis • Critical bottleneck identified: System highlights urgent attention needed • Administrator requests custom analysis: System provides configuration options
Special Requirements	• Dashboard must refresh data at least hourly • Analytics must be available for individual positions and aggregate across company • System must maintain data privacy in accordance with regulations

Table 2.11: Use Case 11: Generate Recruitment Analytics

2.2 Functional Requirements

2.2.1 Module 1

Following are the requirements for module 1:

1. FR-1.1: The system shall parse uploaded resumes with 95% accuracy for extracting key professional information.
2. FR-1.2: The system must support resume uploads in PDF and DOCX formats.
3. FR-1.3: The system shall generate optimization suggestions tailored to industry-specific standards.
4. FR-1.4: Resume processing and analysis must complete within 30 seconds of upload.
5. FR-1.5: The system shall perform multi-factor matching between resume skills and job requirements.
6. FR-1.6: The system must detect potential resume misrepresentations or inconsistencies.
7. FR-1.7: Matching algorithms shall consider at least 10 different professional criteria.

2.2.2 Module 2

Following are the requirements for module 2:

1. FR-2.1: The system shall generate customized assessments based on specific job requirements.
2. FR-2.2: Assessment generation must support multiple programming languages and question types.
3. FR-2.3: The system shall implement adaptive difficulty levels matching job seniority.
4. FR-2.4: Assessment question sets must be automatically updated quarterly with industry-relevant content.
5. FR-2.5: The system shall provide auto-save functionality during skills assessments.

6. FR-2.6: Assessment scoring must complete within 2 minutes of submission.
7. FR-2.7: The system must support multiple assessment formats including multiple-choice, coding, and text responses.
8. FR-2.8: The assessment system shall monitor and flag potential cheating behaviors.

2.2.3 Module 3

Following are the requirements for module 3:

1. FR-3.1: The system shall conduct automated technical and behavioral interviews.
2. FR-3.2: The interview system shall dynamically adapt questions based on previous candidate responses.
3. FR-3.3: Interview sessions must be recorded and stored securely for company review.
4. FR-3.4: Real-time analysis of candidate responses shall evaluate both technical accuracy and communication effectiveness.
5. FR-3.5: The system must maintain a natural conversation flow during automated interviews.
6. FR-3.6: Interview analysis must complete within 10 minutes of session completion.

2.2.4 Module 4

Following are the requirements for module 4:

1. FR-4.1: The system shall generate comprehensive recruitment funnel visualizations.
2. FR-4.2: The system shall analyze and score effectiveness of different recruitment screening stages.
3. FR-4.3: Recruitment analytics must refresh data at least hourly.
4. FR-4.4: The system shall generate process improvement recommendations based on historical data.
5. FR-4.5: The system must implement bias detection mechanisms in candidate evaluation.
6. FR-4.6: Performance benchmarking shall compare against industry standards.

2.2.5 Module 5

Following are the requirements for module 5:

1. FR-5.1: The system shall maintain user status updates within 1 hour of any change.
2. FR-5.2: Multiple notification channels must be supported, including email and in-app notifications.
3. FR-5.3: Critical system interactions must complete within specified performance thresholds:
   (a) Job search results: Load within 3 seconds
   (b) Candidate dashboard: Load within 2 seconds
   (c) Job listing publishing: Complete within 5 seconds
4. FR-5.4: All system modules must implement auto-save functionality.
5. FR-5.5: The platform shall ensure compatibility across multiple devices and browsers.
6. FR-5.6: User authentication and profile management must comply with data privacy regulations.
7. FR-5.7: The system shall implement secure mechanisms to prevent unauthorized access.

2.3 Non-Functional Requirements

2.3.1 Reliability

REL-1: The system shall maintain a Mean Time Between Failures (MTBF) of at least 720 hours (30 days) of continuous operation.

REL-2: In the event of a system failure, no more than 0.1% of ongoing candidate assessments or interviews shall be irretrievably lost.

REL-3: The system must implement comprehensive error logging and tracking mechanisms to capture all critical system failures.

REL-4: Automatic recovery procedures shall be initiated within 30 seconds of detecting a system anomaly.

REL-5: Critical data (user profiles, ongoing assessments, interview recordings) must be backed up in real-time with redundant storage.

REL-6: The system shall provide clear error messages that guide users or administrators on potential recovery actions.

2.3.2 Usability

USE-1: The user interface shall be navigable with no more than three clicks to access any primary system function.

USE-2: The system must support accessibility standards, including full keyboard navigation and screen reader compatibility.

USE-3: First-time users shall achieve basic task completion (resume upload, job application) with 90% success rate within 10 minutes of initial interaction.

USE-4: Context-sensitive help and tooltips must be available for all complex system interactions.

USE-5: The user interface shall maintain consistent design language across all modules, with intuitive iconography and layout.

USE-6: System shall support multiple languages, with at least 5 major international languages fully localized.

USE-7: Error messages must be written in clear, non-technical language that guides users toward resolution.

2.3.3 Performance

PER-1: 95% of all system interactions shall complete within 2 seconds under normal load conditions.

PER-2: The system must support concurrent operations for at least 10,000 active users without performance degradation.

PER-3: Resume parsing and optimization shall complete within 30 seconds for documents up to 10 pages long.

PER-4: Job search results must load within 3 seconds, displaying at least 50 relevant job listings.

PER-5: AI interview and assessment systems shall provide real-time response analysis with less than 500 milliseconds latency.

PER-6: Data visualization and analytics reports must generate within 5 seconds for datasets up to 10,000 records.

PER-7: System shall maintain consistent performance across desktop, tablet, and mobile platforms.

2.3.4 Security

SEC-1: The system must prevent unauthorized access with multi-factor authentication mechanisms.

SEC-2: All sensitive personal and professional data shall be encrypted at rest and in transit using industry-standard encryption protocols.

SEC-3: The system shall implement comprehensive audit logging for all critical user actions and administrative functions.

SEC-4: User authentication attempts shall be limited to 5 consecutive failed attempts before temporary account lockout.

SEC-5: Personal identification information must be protected with role-based access controls.

SEC-6: The system shall automatically detect and flag suspicious user behavior patterns.

SEC-7: Compliance with GDPR, CCPA, and other relevant data protection regulations must be maintained.

SEC-8: All system integrations and third-party data exchanges must use secure, authenticated API connections.

2.3.5 Scalability

SCA-1: The system architecture must support horizontal scaling to accommodate 50% user base growth without performance degradation.

SCA-2: Database operations shall maintain consistent performance with data volumes increasing by up to 100% annually.

SCA-3: Microservices architecture must allow independent scaling of individual system components.

2.3.6 Compatibility

COM-1: The system shall be fully functional on the following platforms:

- Web browsers: Chrome, Firefox, Safari, Edge (latest two versions)
- Desktop operating systems: Windows, macOS, Linux

2.3.7 Maintainability

MAI-1: System code shall maintain a documentation coverage of 90% for all critical modules.

MAI-2: Automated testing must provide code coverage of at least 85% across all system components.

MAI-3: System updates and patches must be deployable with zero downtime.

Chapter 3

System Overview

InterviewDen addresses critical recruitment challenges through an innovative technological solution. The platform transforms talent acquisition by providing advanced capabilities for both organizations and job seekers. For candidates, the system offers sophisticated profile optimization and precise job matching beyond traditional screening methods. Organizations benefit from a streamlined workflow that delivers comprehensive insights into candidate potential, reducing hiring cycle times and improving selection quality. By implementing advanced analytical methodologies, InterviewDen mitigates traditional hiring biases and provides a holistic assessment of professional capabilities. The platform represents a strategic approach to connecting organizational needs with professional talent through intelligent, data-driven solutions.

3.1 Architectural Design

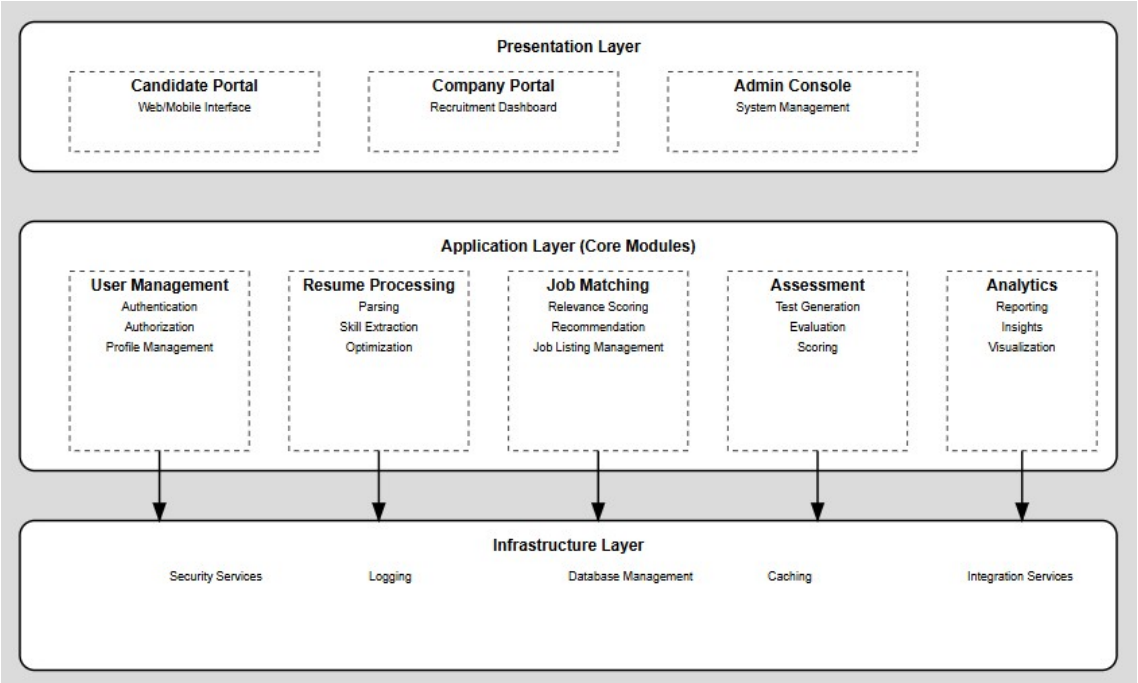

Figure 3.1: System Architecture

The system architecture diagram depicts InterviewDen's comprehensive technology platform, structured across three primary layers. The Presentation Layer shows user interfaces for candidates, companies, and administrators, providing entry points into the system. The Application Layer forms the core of the platform, with five interconnected modules: User Management, Resume Processing, Job Matching, Assessment, and Analytics. Each module has specialized functions that work together to streamline the recruitment process. The bottom Infrastructure Layer supports these modules with essential services like security, logging, database management, caching, and integration capabilities. This modular design allows for flexible, scalable, and efficient talent acquisition technology that can adapt to diverse recruitment needs while maintaining robust performance and interconnectivity.

3.2 Data Design

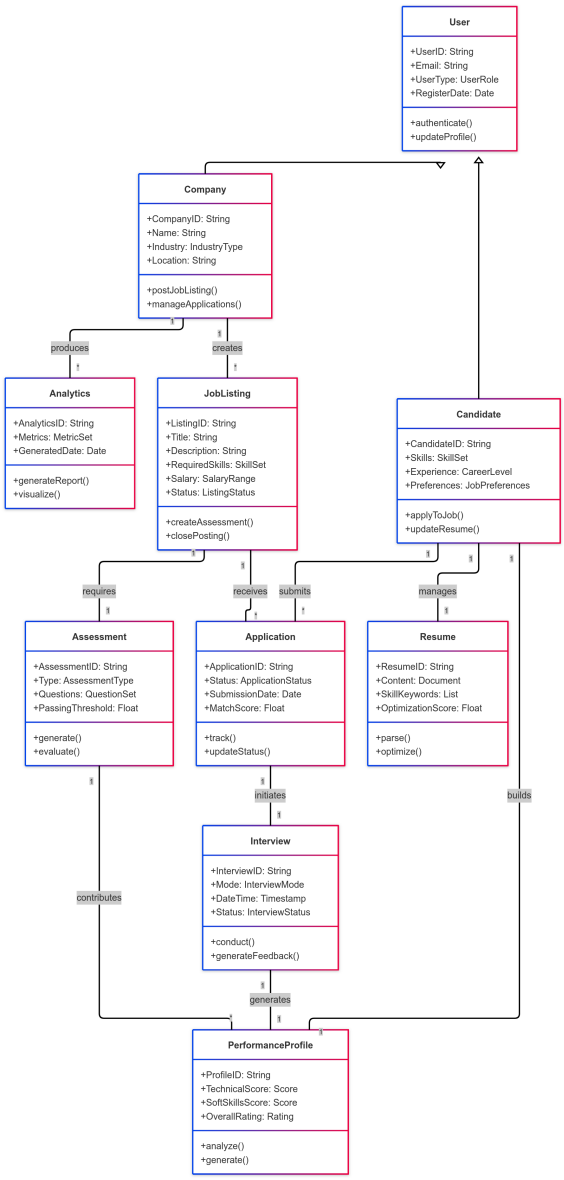

Figure 3.2: Data Design Diagram

3.3 Domain Model

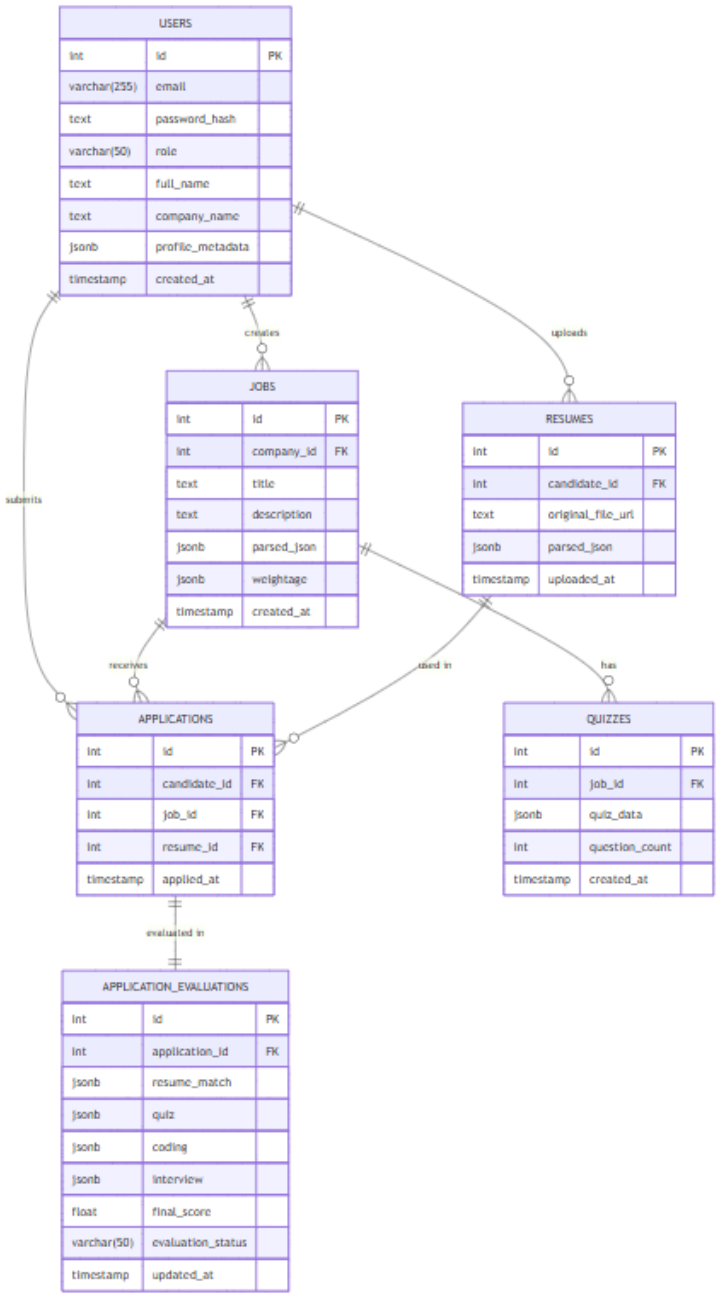

Figure 3.3: Domain Model Diagram

3.4 Design Models [UPTO THE CURRENT ITERATION ONLY]

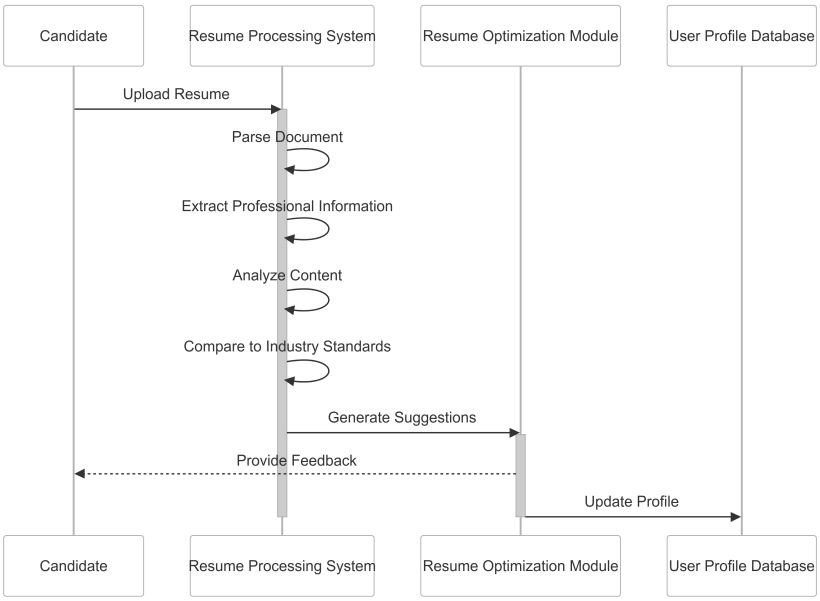

Figure 3.4: SSD 1

Chapter 4

Implementation and Testing [UPTO THE CURRENT ITERATION ONLY]

4.1 Algorithm Design

The resume screening module employs a hybrid approach combining structured parsing with LLM-based contextual analysis to evaluate candidate resumes against job requirements.

4.1.1 Data Processing Pipeline

The resume matching algorithm operates through a multi-stage pipeline:

1. **Document Conversion**: We transform unstructured documents (PDF resumes and text job descriptions) into structured JSON format for algorithmic processing.

2. **Information Extraction**: The system identifies and extracts key entities from both documents:

    - From resumes: skills, experience, education, certifications, and projects
    - From job descriptions: required skills, preferred skills, experience requirements, and qualifications

3. **Hierarchical Matching**: The algorithm performs matching at multiple levels of abstraction to ensure comprehensive evaluation.

4.1.2 Core Matching Algorithm

Our matching approach combines rule-based matching with LLM-enhanced semantic understanding:

4.1.2.1 Hard Facts Matching

The system first performs exact matching on quantifiable elements:

4.1.2.2 LLM-Enhanced Contextual Analysis

To overcome limitations of exact matching, we employ the LLM to perform contextual analysis:

4.1.2.3 Composite Scoring

The final score combines both approaches to leverage the strengths of each:

4.1.3 Algorithm Advantages

This hybrid approach offers several key advantages:

1. **Objectivity with Nuance**: The hard facts matching ensures objective evaluation of explicit requirements, while the LLM component captures nuanced skills relationships and transferable experience.
2. **Semantic Understanding**: The LLM can recognize equivalent skills (e.g., understanding that "React" experience is relevant for a "Frontend JavaScript Framework" requirement) that would be missed by exact matching.
3. **Context-Aware Evaluation**: The algorithm considers the context of skills and experience, not just their presence or absence (e.g., distinguishing between a candidate with 5 years of relevant Python experience versus one with only 6 months).
4. **Balanced Assessment**: By combining rule-based and AI-powered approaches, the algorithm achieves a balance between consistent objective evaluation and intelligent contextual understanding.
5. **Valuable Feedback Generation**: The structured output includes not just match percentages but also specific strengths and improvement areas that provide actionable feedback for candidates.

4.1.4 Implementation Considerations

In implementing this algorithm, we addressed several technical challenges:

1. **Prompt Engineering**: Carefully crafted LLM prompts to ensure consistent, structured responses suitable for programmatic processing.
2. **Error Handling**: Implemented fallback mechanisms to rely on hard facts matching if LLM service is unavailable or produces malformed responses.
3. **Score Calibration**: Adjusted weighting parameters based on empirical testing to ensure scores align with expert human evaluations.
4. **Processing Efficiency**: Optimized document processing to maintain acceptable response times even with complex resumes and detailed job descriptions.

This algorithm design forms the core of our resume screening module, enabling accurate, nuanced matching while maintaining computational efficiency and robustness.

```python
def read_job_description(file_path: str) -> str:
    """
    Read job description from a text file

    Args:
        file_path: Path to the job description text file

    Returns:
        Job description text as a string
    """
    if not os.path.exists(file_path):
        raise FileNotFoundError(f"Job description file not found at: {file_path}")

    with open(file_path, 'r', encoding='utf-8') as f:
        text = f.read()

    return text

def process_job_description_with_llm(job_description_text: str) -> Dict[str, Any]:
    """
    Process job description text with Groq's Llama3-70b to structure it into JSON format

    Args:
        job_description_text: Plain text job description

    Returns:
        Structured job description data as a dictionary
    """
    # Check cache first - create hash of input text for cache key
    import hashlib
    cache_key = hashlib.md5(job_description_text.encode()).hexdigest()
    cache_dir = os.path.join("temp", "cache")
    os.makedirs(cache_dir, exist_ok=True)
    cache_file = os.path.join(cache_dir, f"job_{cache_key}.json")

    # If cache exists, return cached result
    if os.path.exists(cache_file):
        print("Using cached job description result...")
        with open(cache_file, 'r', encoding='utf-8') as f:
            return json.load(f)

    # Get API key from config file
    api_key = config.GROQ_API_KEY
```

Figure 4.1: Converting Job Description to JSON

```python
def match_resume_job(args):
    """Compare a resume with a job description and evaluate the match"""
    # Validate resume JSON path
    resume_file = args.resume
    if not os.path.exists(resume_file):
        print(f"Error: Resume JSON file not found at {resume_file}")
        return

    # Validate job description JSON path
    job_file = args.job
    if not os.path.exists(job_file):
        print(f"Error: Job description JSON file not found at {job_file}")
        return

    try:
        print(f"Loading resume from: {resume_file}")
        print(f"Loading job description from: {job_file}")

        # Evaluate the match with thresholds
        evaluation = resume_match.evaluate_match(
            resume_file,
            job_file,
            args.output,
            args.overall_threshold,
            args.skill_threshold,
            args.experience_threshold
        )

        print(f"Match evaluation saved to: {args.output}")

        # Print a summary
        print("\nMatch Evaluation Summary:")
        print(f"Overall Match Score: {evaluation.get('match_score', 'N/A')}/100")
        print(f"Skill Match Score: {evaluation.get('skill_match_score', 'N/A')}/100")
        print(f"Experience Match Score: {evaluation.get('experience_match_score', 'N/A')}/100")
```

Figure 4.2: Resume Matching with Job Description

```python
def process_resume_with_llm(resume_text: str, job_description: str = None) -> Dict[str, Any]:
    """
    Process resume text with Groq's Llama3-70b to structure it into JSON format

    Args:
        resume_text: Plain text extracted from resume
        job_description: Optional job description text for context

    Returns:
        Structured resume data as a dictionary
    """
    # Check cache first - create hash of input text and job description for cache key
    cache_key_content = resume_text
    if job_description:
        cache_key_content += job_description

    cache_key = hashlib.md5(cache_key_content.encode()).hexdigest()
    cache_dir = os.path.join("temp", "cache")
    os.makedirs(cache_dir, exist_ok=True)
    cache_file = os.path.join(cache_dir, f"resume_{cache_key}.json")

    # If cache exists, return cached result
    if os.path.exists(cache_file):
        print("Using cached resume result...")
        with open(cache_file, 'r', encoding='utf-8') as f:
            return json.load(f)

    # Get API key from config file
    api_key = config.GROQ_API_KEY

    if not api_key:
        raise ValueError("Groq API key is not set. Please update the GROQ_API_KEY in config.py")

    # System message to define the task for the LLM
    system_message = """
    You are a resume parsing assistant. Extract and structure information from the resume text into the following JSON format:

    {
      "Name": "",
      "Contact": {
        "Email": "",
        "Number": ""
```

Figure 4.3: Resume Score Criteria

```python
{
  "job_title": "Remote Fullstack Developer",
  "role_description": "Develop AI-powered applications for commercial and research-driven projects, working alongside AI specialists to push the boundaries of innovation.",
  "experience_required": {
    "years_of_experience": "Not specified"
  },
  "skills_required": {
    "technical_skills": [
      "Python",
      "JavaScript",
      "TypeScript"
    ],
    "soft_skills": [
      "Strong analytical skills",
      "Ability to troubleshoot complex software issues"
    ]
  },
  "preferred_skills": [],
  "job_responsibilities": [
    "Design and develop full-stack applications utilizing Python, JavaScript, and TypeScript.",
    "Architect scalable and efficient systems to support AI-driven functionalities.",
    "Work closely with AI researchers and data scientists to integrate machine learning models into applications.",
    "Debug, document, and enhance software solutions to ensure optimal user experience.",
    "Collaborate with cross-functional teams to transform business needs into technical solutions."
  ]
}
```

Figure 4.4: Job Description JSON

```python
{
  "Name": "ASAD ULLAH SHAH",
  "Contact": {
    "Email": "asadullahshah002@gmail.com",
    "Number": "+92 335 5222247"
  },
  "Experience": {
    "Total Years": 0.5,
    "Experiences": [
      {
        "Title": "FULL STACK WEB DEVELOPER",
        "Company": "Blunder Bot Technologies",
        "Duration": "October 2021 - Present",
        "Description": "Executed a cutting-edge, futuristic UI/UX design for a weather application, showcasing exceptional skills and innovative approach to UI/UX design."
      },
      {
        "Title": "SENIOR DEVELOPER",
        "Company": "Rushify",
        "Duration": "November 2023- Present",
        "Description": "Created an aesthetically pleasing and user-friendly static website using Framer (No-Code/Low-Code framework)"
      }
    ]
  },
  "Education": [
    {
      "Degree": "BACHELOR OF SCIENCE IN COMPUTER SCIENCE",
      "University": "NUCES",
      "Duration": "September 2021 - June 2025"
    }
  ],
  "Skills": [
    "Python",
    "Javascript",
    "Java",
    "C/C++",
    "C#",
    "SQL Server",
```

Figure 4.5: Resume JSON

```
{
  "match_score": 86,
  "skill_match_score": 84,
  "experience_match_score": 88,
  "comment": [
    "Strong skill experience with Typescript, a required technical skill.",
    "Insufficient evidence of experience with AI-powered applications or machine learning models.",
    "UI/UX design skills are mentioned, but not directly relevant to the job description."
  ],
  "missing_skills": [
    "Typescript"
  ],
  "overqualified_bf": [],
  "gaps_bf": [],
  "status": "Hold",
  "failed_criteria": [
    "Overall match score (86) below minimum threshold (70)",
    "Skill match score (84) below minimum threshold (85)",
    "Experience match score (88) below minimum threshold (90)"
  ],
  "feedback_message": "Thank you for your interest in this position. After careful review of your application, we've identified some areas for improvement that would strengthen your alignment with this role's requir
}
```


Figure 4.6: Resume Score JSON

4.2 External APIs/SDKs

API and version	Description	Purpose of usage	API endpoint/function/class used
GROQ API	LLM based analysis	For Resume Score	api.groq.com/openai/v1/chat/completions

4.3 Testing Details

4.3.1 Unit Testing

We designed and implemented systematic unit tests for all planned core components:

- **ResumeParser**: Validated mechanisms for extracting text from PDFs, identifying resume sections (skills, experience, education), and handling various resume formats.
- **JobDescriptionParser**: Confirmed methods for extracting job titles, required and preferred skills, experience requirements, and education criteria from unstructured job descriptions.
- **ResumeMatcher**: Tested algorithms for calculating match percentages, identifying matching and missing skills, and appropriately weighting different factors in the overall score.
- **LLM Integration**: Verified approaches for communicating with the language model service, constructing effective prompts, and handling potential service interruptions.

4.3.2 Integration Testing

We designed tests for the end-to-end resume screening workflow to ensure:

- Seamless data flow between components
- Proper transformation of PDF inputs to structured match results
- Consistent database storage of matching outcomes
- Appropriate error handling across component boundaries

4.3.3 Prototype Validation

4.3.3.1 Diverse Resume-Job Testing

To validate our approach before full implementation, we conducted controlled testing with representative samples:

- **Test Dataset**: We compiled a focused test set of 10 resumes spanning different quality levels (high, medium, low) and 10 job descriptions across various technical roles (frontend, backend, data science, DevOps, QA).
- **Matching Combinations**: We tested key resume-job combinations, covering scenarios from perfect matches to completely misaligned skills.
- **Expected Outcome Validation**: For each combination, we verified that our proposed matching algorithms generated scores within predetermined expected ranges based on the known quality of the match.

4.3.3.2 Algorithm Effectiveness Assessment

To ensure our algorithmic approach would align with human judgment:

- **Reference Comparisons**: We compared our algorithm's output against established resume-screening benchmarks from academic literature and industry standards.
- **Edge Case Examination**: We identified potential limitations in standard matching approaches, particularly in evaluating transferable skills and equivalent technologies.

Bibliography

Appendix A

Appendices

A.1 Appendix A

A.1.1 Use Case Diagram

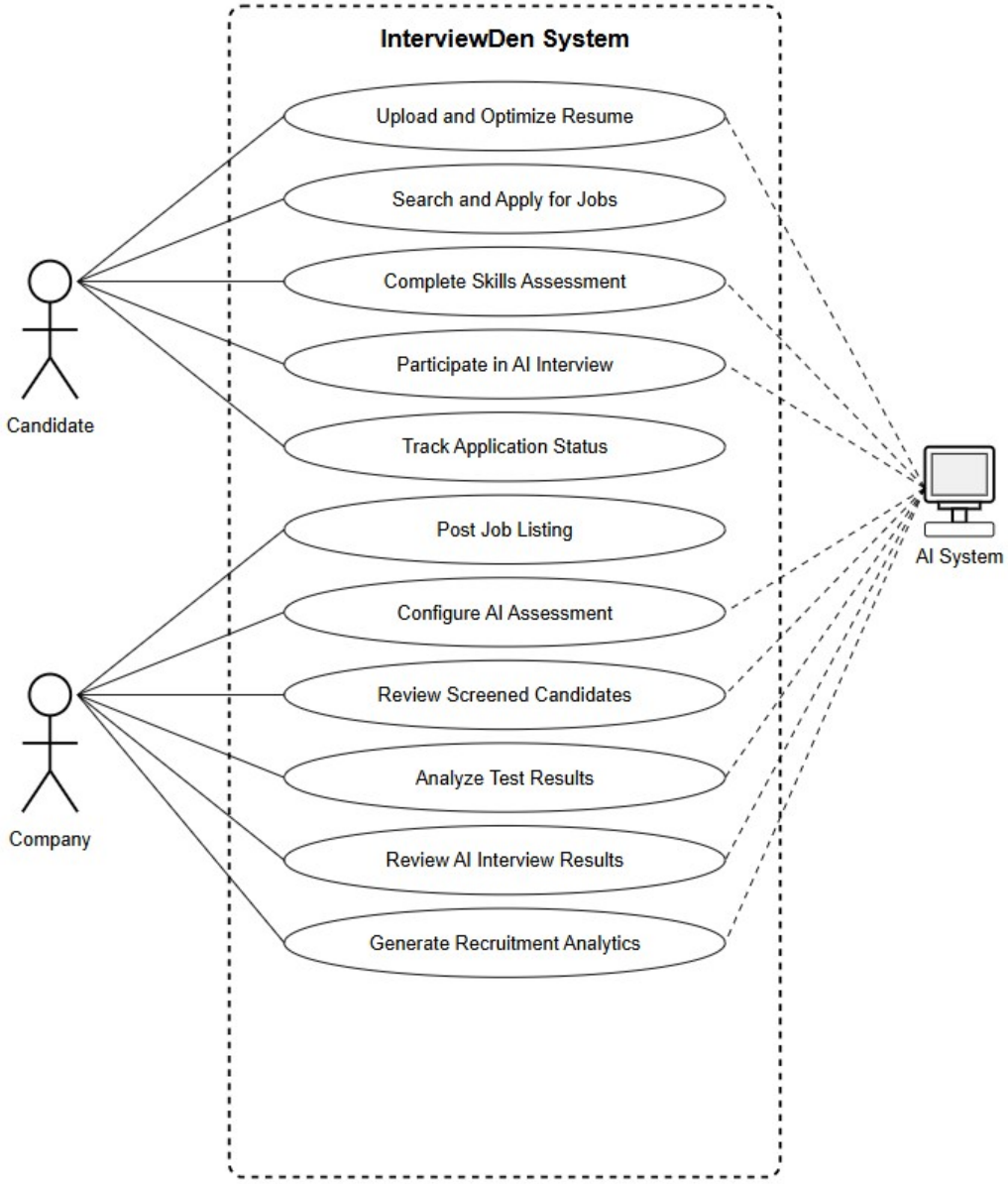

Figure A.1: Use Case Diagram

A.2 Appendix B

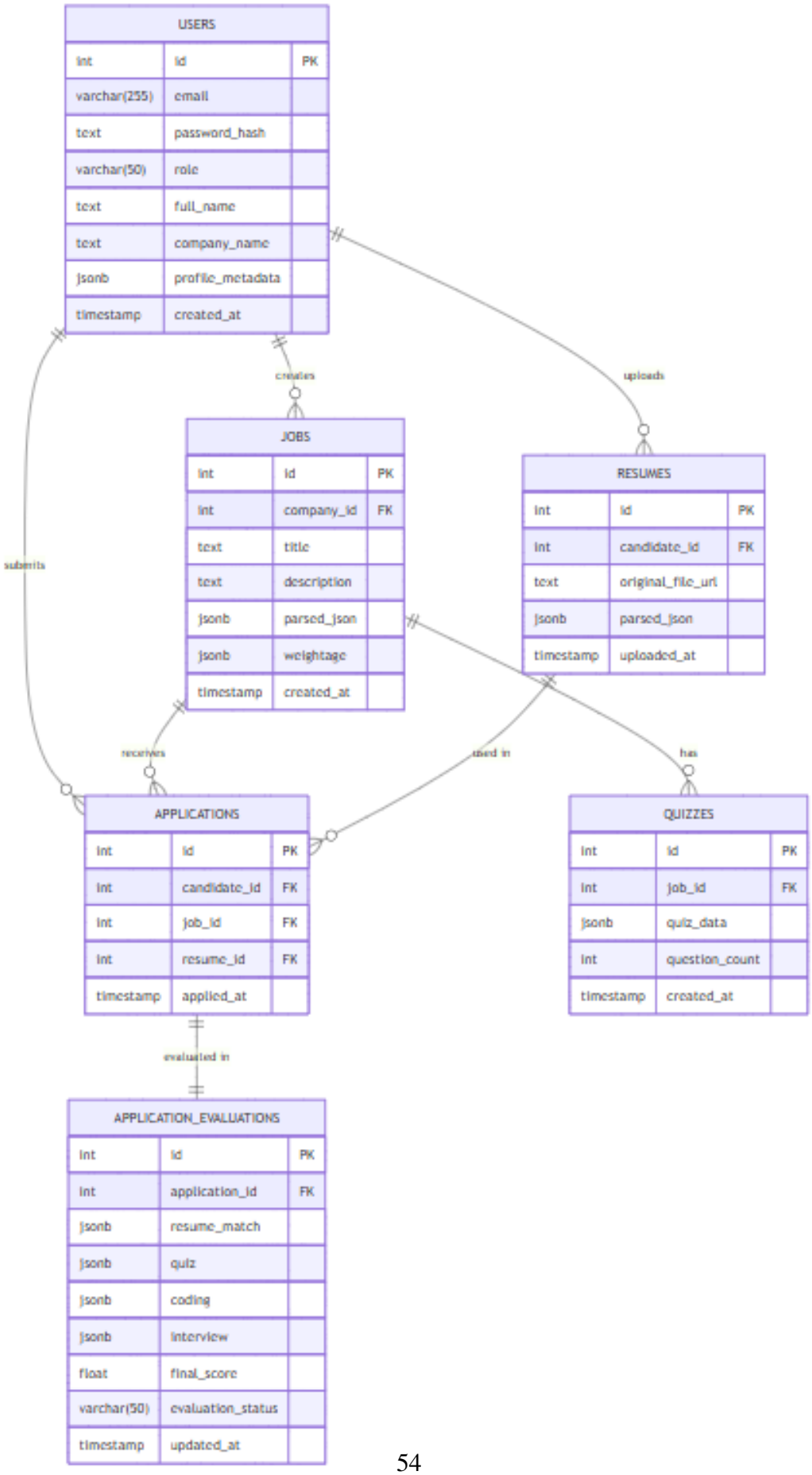

Figure A.2: Domain Model Diagram

A.3 Appendix C

A.3.1 Architecture Design

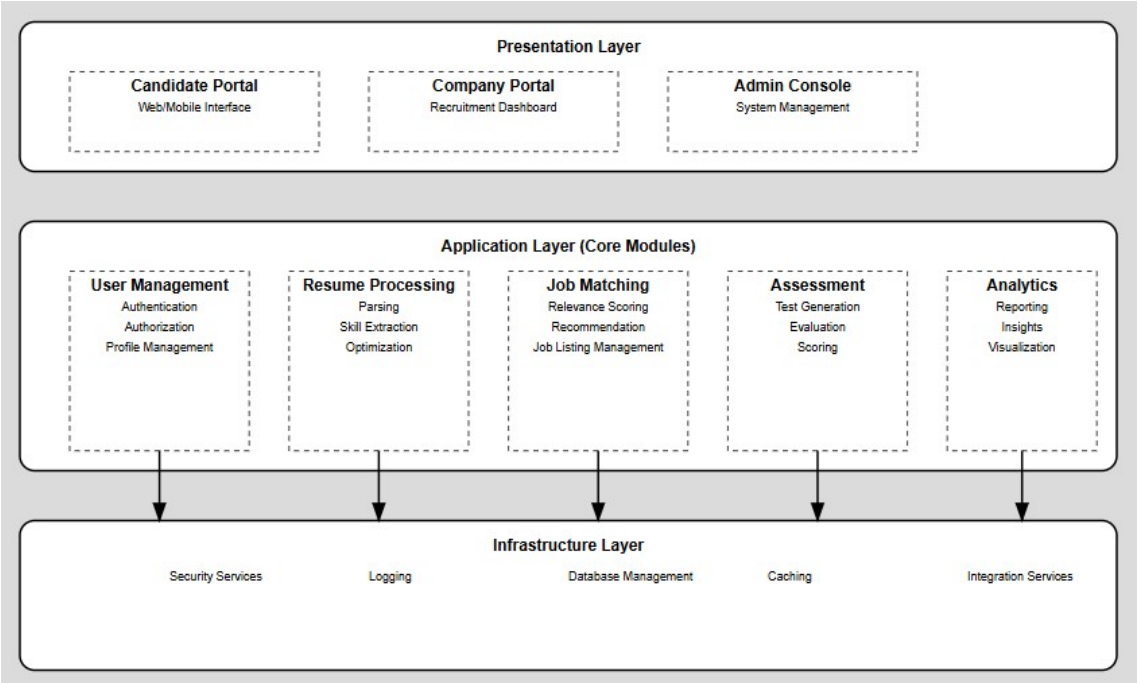

Figure A.3: Architecture Diagram

A.4 Appendix D

A.4.1 System Sequence Diagram

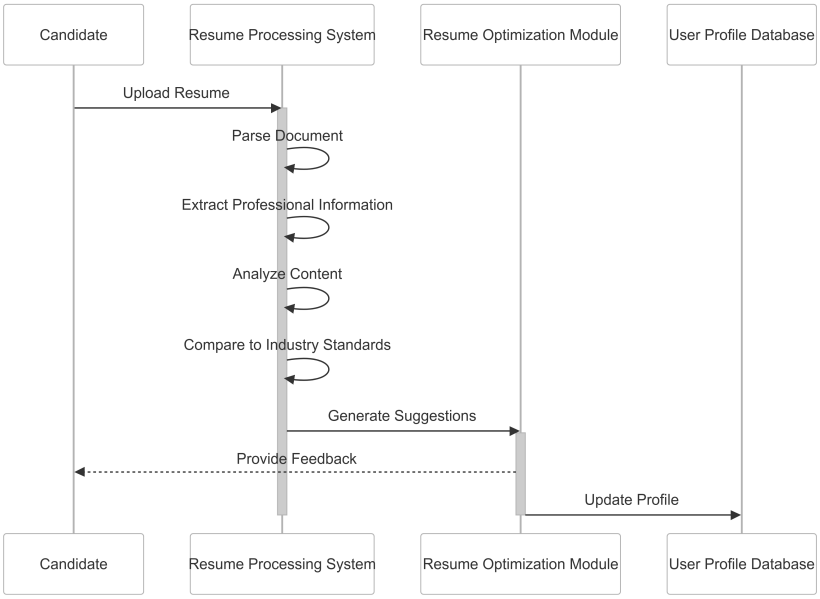

Figure A.4: SSD 1

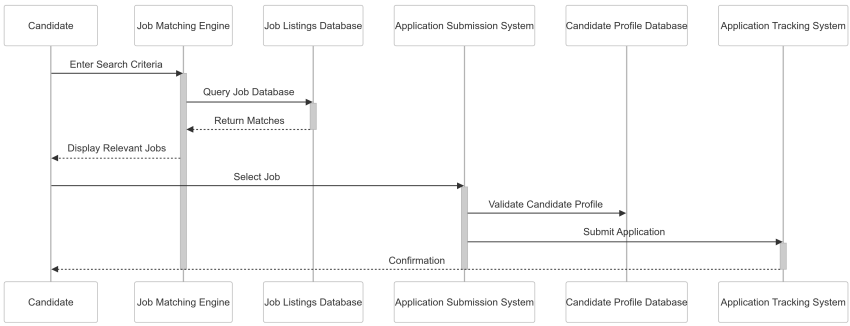

Figure A.5: SSD 2

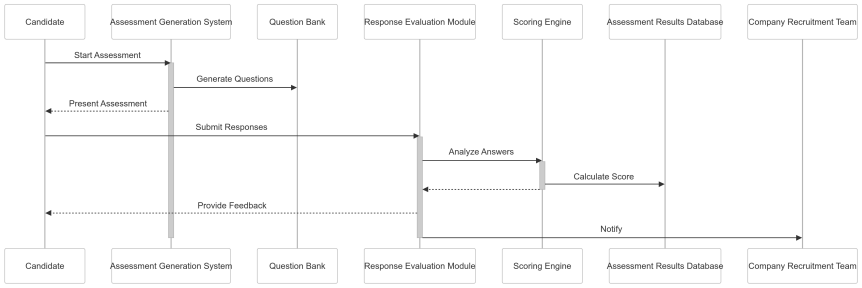

Figure A.6: SSD 3

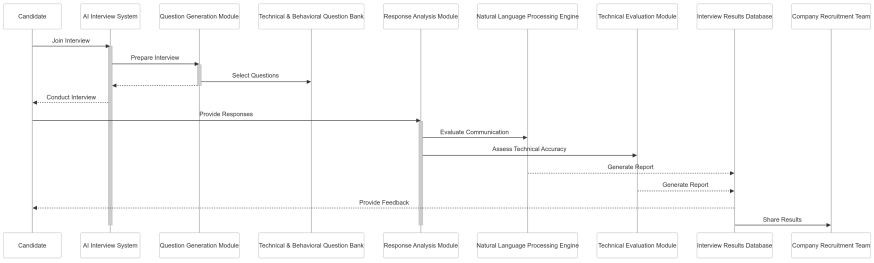

Figure A.7: SSD 4

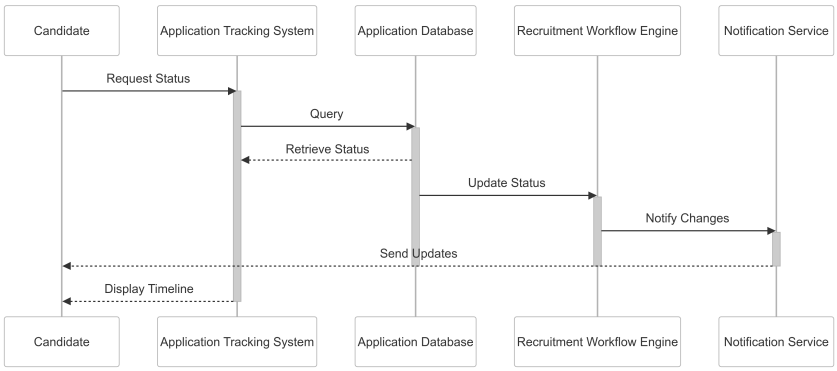

Figure A.8: SSD 5

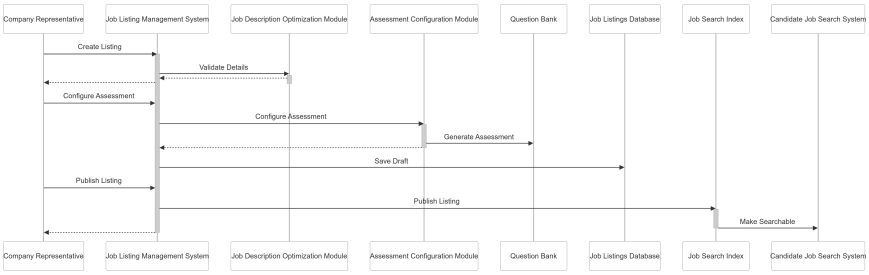

Figure A.9: SSD 6

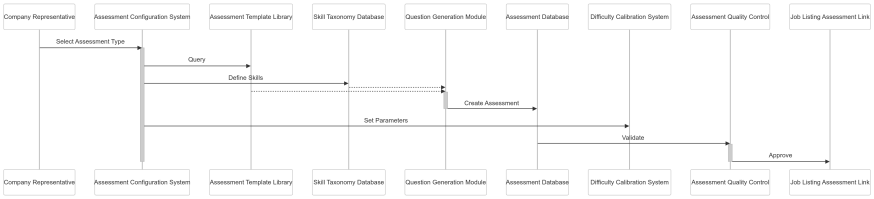

Figure A.10: SSD 7

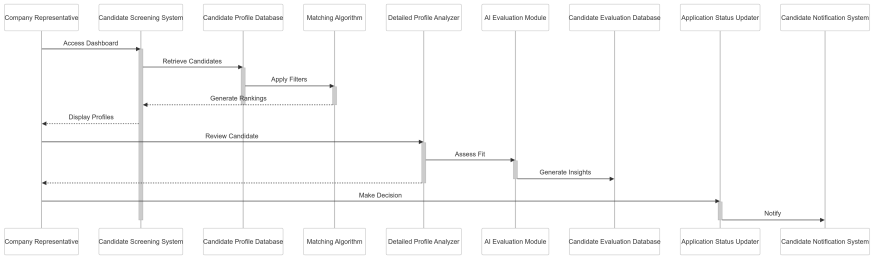

Figure A.11: SSD 8

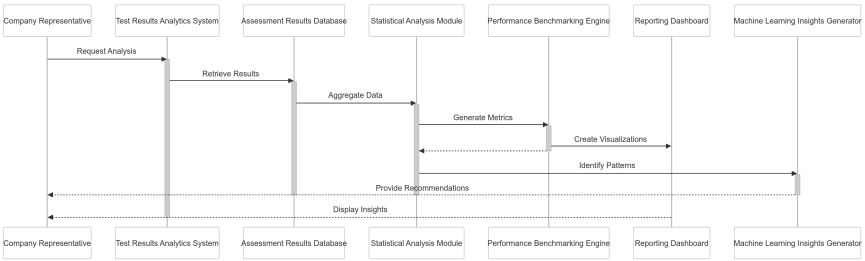

Figure A.12: SSD 9

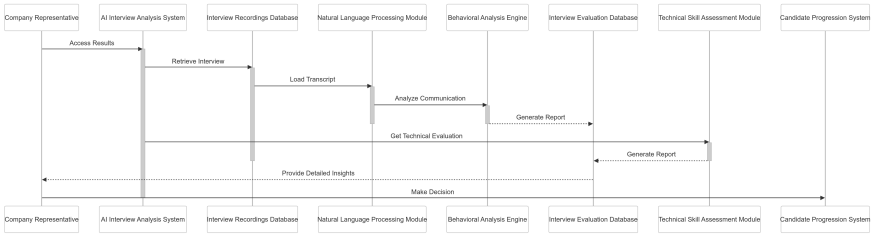

Figure A.13: SSD 10

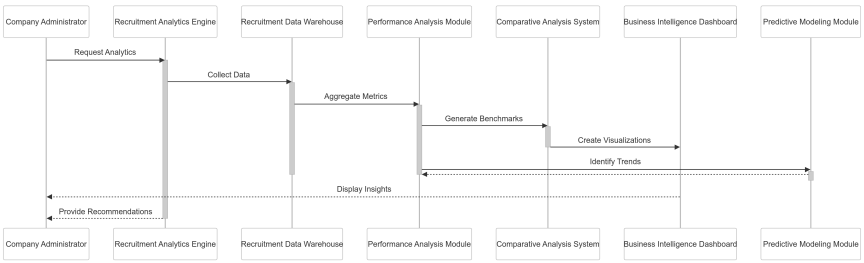

Figure A.14: SSD 11